

College Complaint Management System

A full-stack cross-platform mobile and web application for digitized student grievance redressal

COMPLETE PROJECT ANALYSIS & TEAM CONTRIBUTION REPORT

Four-Member Team Division • Architecture • Features • Security • Improvements

FRONTEND	BACKEND	DATABASE	DATE
Expo React Native	Spring Boot 4.1	MySQL (Aiven)	October 2026

Table of Contents

01. Project Overview

02. Features and Functionalities

03. Frontend Analysis

04. Backend Analysis

05. Database Analysis

06. Complete System Architecture & Workflow

07. Four-Member Team Division

08. Project Folder Structure

09. Security and Configuration

10. Missing Features and Improvements

11. Final Project Summary

1 Project Overview

Project Title & Purpose

The project is titled **College Complaint Management System** (package: `com.project.collegecomplaintapp` / `ComplaintApp`). It is a full-stack cross-platform mobile and web application designed to streamline, digitize, and monitor student grievance redressal at **Sarvajanik College of Engineering & Technology (SCET)**.

Problem the Project Solves

In traditional college environments, student complaints are handled through manual paper forms, physical suggestion boxes, or unorganized email threads. This leads to lost requests, zero transparency on resolution progress, missed deadlines without accountability, and administrative bottlenecks. This system solves these problems by providing:

- A single authenticated mobile/web hub for filing and tracking complaints with photographic evidence.
- Automated SLA deadline enforcement based on college working hours (9 AM – 5 PM, Monday–Friday).
- Automated multi-channel email notifications (OTP verification, Status Updates, SLA Breach alerts).
- An executive administrative dashboard with status triage, multi-criteria filtering, and audit remarks.

Target Users

USER ROLE	DESCRIPTION
STUDENT	Enrolled college students holding institutional email addresses (`@scet.ac.in`) who submit complaints, track resolution stages, attach evidence, and manage profiles.
ADMIN	Department heads, maintenance officers, and grievance committee members who review issues, triage priorities, progress complaints through lifecycle states, add official remarks, and inspect photographic proof.

Technologies & Tools Used

LAYER	TECHNOLOGY	DETAILS
Frontend	Expo React Native	React 19.2.3, React Native 0.86.3, Expo SDK 57, Expo Router, TypeScript 6.0

LAYER	TECHNOLOGY	DETAILS
Frontend Libraries	Various	expo-image-picker, expo-secure-store, AsyncStorage, react-native-reanimated, react-native-svg, react-native-element-dropdown, @expo/vector-icons
Backend	Spring Boot 4.1.0	Java 21, Spring WebMVC, Spring Data JPA, Spring Security, Spring Mail, JJWT 0.12.5, Lombok
Database	MySQL 8.x	Managed instance hosted on Aiven Cloud with SSL
Cloud Storage	Supabase Storage	REST API for `profiles` and `complaints` buckets
Email Services	Multi-Provider	Brevo REST API, Resend REST API, Jakarta Mail / SMTP
Deployment	Render / Docker	Multi-stage Docker build, Render Cloud Web Service, EAS Build for mobile APK

Main Objectives & Key Benefits

- **Institutional Domain Gating**: Restricts account creation strictly to official college domain emails (`@scet.ac.in`) with OTP verification.
- **End-to-End Transparency**: Provides live tracking across stages: SUBMITTED → ASSIGNED → IN_PROGRESS → RESOLVED / REJECTED.
- **Business-Hours SLA Deadline Engine**: Automatically calculates realistic deadlines accounting for evenings and weekends.
- **Cloud-Backed Media Handling**: Uploads photos directly to Supabase cloud buckets with CDN URLs.

2 Features and Functionalities

Authentication & Identity

FEATURE	DESCRIPTION	KEY FILES	STATUS
Institutional Email OTP Registration	Validates college email (`@scet.ac.in`), sends 6-digit cryptographic OTP valid for 5 min, verifies it, encodes password with BCrypt, and registers student with optional profile photo.	`register.tsx` , `AuthController.java` , `AuthService.java` , `EmailOtp.java`	FULLY IMPLEMENTED
JWT Role-Based Login	Authenticates credentials, generates a 24-hour signed HMAC-SHA256 JWT containing userId and role, returning user metadata for role-based routing.	`login.tsx` , `AuthController.java` , `JwtUtil.java` , `SecurityConfig.java`	FULLY IMPLEMENTED
Password Reset via OTP	Allows registered students to request a 6-digit reset OTP via email and securely update their password.	`forgotPassword.tsx` , `AuthController.java` , `AuthService.java`	FULLY IMPLEMENTED
Animated Splash & Session Guard	Animated branded launch screen with SVG circular progress loader verifying existing credentials and routing to either Student or Admin portal.	`index.tsx` , `authStorage.ts`	FULLY IMPLEMENTED

Student Grievance Flow

FEATURE	DESCRIPTION	KEY FILES	STATUS
Complaint Submission with Photo	Multi-field form for lodging a complaint with dynamic category dropdown, priority selection, form validation, camera/gallery	`addComplaint.tsx` , `ComplaintController.java` , `ComplaintService.java`	FULLY IMPLEMENTED

FEATURE	DESCRIPTION	KEY FILES	STATUS
	attachment, and multipart upload to Supabase.		
Student Dashboard & KPI Metrics	Aggregates student-specific complaints, calculates metrics (Total, Submitted, In Progress, Resolved), and displays recent issues with profile info.	`dashboard.tsx`, `ComplaintController.java`, `UserContoller.java`	FULLY IMPLEMENTED
Personal Grievance History	Lists student's submitted grievances sorted by status priority and creation date, with image view toggles, SLA breach badges, and admin remarks display.	`myComplaints.tsx`, `ComplaintRepository.java`	FULLY IMPLEMENTED
User Profile & Avatar	Shows user academic metadata and enables live camera/photo-library avatar updating stored directly on Supabase cloud storage.	`profile.tsx`, `UserContoller.java`, `SupabaseStorageService.java`	FULLY IMPLEMENTED

Admin Operations

FEATURE	DESCRIPTION	KEY FILES	STATUS
Executive Triage Dashboard	Summarizes institutional statistics (Total, Submitted, Assigned, In Progress, Resolved, Rejected), with quick links and recent activity feed.	`admin/dashboard.tsx`, `AdminController.java`, `AdminService.java`	FULLY IMPLEMENTED
Complaint Management List	Searchable and filterable master view of all student complaints across departments, categories, priorities, and statuses with pagination support.	`admin/complaints.tsx`, `AdminController.java`, `AdminService.java`	FULLY IMPLEMENTED

FEATURE	DESCRIPTION	KEY FILES	STATUS
Status Lifecycle Machine	Allows admins to step complaints forward (SUBMITTED → ASSIGNED → IN_PROGRESS → RESOLVED) or REJECT with mandatory remarks. Sends email notification on every transition.	`admin/complaintDetails.tsx`, `AdminController.java`, `AdminService.java`	FULLY IMPLEMENTED

Business Logic & Integration Engine

FEATURE	DESCRIPTION	KEY FILES	STATUS
Working-Hours SLA Deadline	Computes resolution deadlines strictly during Mon–Fri 9 AM – 5 PM. HIGH: 12 working hours, MEDIUM: 24 working hours, LOW: 48 working hours.	`WorkingHoursDeadlineService.java`	FULLY IMPLEMENTED
SLA Breach Scheduler	Background daemon running every 60 seconds that detects overdue unresolved complaints and emails students an escalation alert.	`ComplaintDeadlineScheduler.java`, `EmailService.java`	FULLY IMPLEMENTED
Multi-Provider Email	Failover email service using Brevo REST API → Resend REST API → SMTP JavaMailSender for OTPs, status updates, and SLA breach alerts.	`EmailService.java`	FULLY IMPLEMENTED
Cloud Storage Manager	Uploads and manages profile images and complaint attachments to Supabase Storage buckets via Spring RestClient.	`SupabaseStorageService.java`	FULLY IMPLEMENTED

FEATURE	DESCRIPTION	KEY FILES	STATUS
Dark & Light Theme	Context-driven theme system with persistent theme memory across mobile (AsyncStorage) and web (localStorage).	`ThemeContext.tsx`, `theme.ts`, `ThemeToggle.tsx`	FULLY IMPLEMENTED

3 Frontend Analysis

Framework, Language & Libraries

- **Framework**: Expo SDK 57 on React Native 0.86.3 with React 19.2.3
- **Language**: TypeScript 6.0
- **Routing**: `expo-router` (file-based Stack and Modal routing)
- **Dropdowns**: `react-native-element-dropdown` (department and division pickers)
- **Media**: `expo-image-picker` & `expo-image` (camera capture, gallery selection)
- **Storage**: `expo-secure-store` & `@react-native-async-storage/async-storage`
- **Animations**: `react-native-reanimated` & `react-native-svg`

Main Screens & Navigation

SCREEN FILE	PURPOSE	ROLE
`app/index.tsx`	Animated 6-second SVG splash screen with session verification routing	All
`app/login.tsx`	Student & Admin authentication with JWT persistence	All
`app/register.tsx`	Multi-step student registration with OTP, photo, department pickers	Student
`app/forgotPassword.tsx`	OTP-based self-service password reset	Student
`app/dashboard.tsx`	Student home dashboard with metrics and recent complaints	Student
`app/addComplaint.tsx`	Complaint submission form with camera/gallery & validation	Student
`app/myComplaints.tsx`	Personal complaint history list with SLA indicators	Student
`app/profile.tsx`	Student profile inspection and avatar photo management	Student
`app/admin/dashboard.tsx`	Admin executive metrics and recent activity feed	Admin
`app/admin/complaints.tsx`	Searchable & filterable master complaint list	Admin

SCREEN FILE	PURPOSE	ROLE
`app/admin/complaintDetails.tsx`	Complaint detail view with status transition controls	Admin

API Communication Layer

All frontend–backend communication is handled through a centralized fetch interceptor (`constants/api.ts`):

- **Token Injection**: Automatically retrieves stored JWT via `getToken()` and injects `Authorization: Bearer <token>` header.
- **Dynamic Content-Type**: If payload is `FormData` (photo uploads), the runtime sets the multipart boundary; otherwise, it injects `Content-Type: application/json`.
- **Session Expiry**: On HTTP 401 Unauthorized, clears stored credentials and redirects to `/login`.
- **Base URL Resolution**: Checks `EXPO_PUBLIC_API_BASE` env var → web hostname detection → fallback development IP.

State Management

Built using idiomatic React hooks (`useState`, `useEffect`, `useCallback`, `useMemo`, `useFocusEffect`) paired with React Context for app-wide theme management (`ThemeContext.tsx`). Auth state is stored platform-adaptively via `authStorage.ts` using `expo-secure-store` on native and `localStorage` on web.

4 Backend Analysis

Framework, Language & Dependencies

- **Language**: Java 21
- **Framework**: Spring Boot 4.1.0 (Spring WebMVC, Spring Data JPA, Spring Security, Spring Mail, Spring Validation)
- **JWT Library**: `io.jsonwebtoken` (JJWT) 0.12.5
- **Database Driver**: `mysql-connector-j` (runtime)
- **Boilerplate Reduction**: Project Lombok (`@Data` , `@Getter` , `@Setter`)
- **Build Tool**: Apache Maven with Maven Wrapper (`mvnw`)

Backend Layer Architecture

LAYER	PACKAGE	COMPONENTS
Controllers	`controller/`	AuthController, ComplaintController, AdminController, UserContoller, controller (health)
Services	`services/`	AuthService, ComplaintService, AdminService, EmailService, SupabaseStorageService, WorkingHoursDeadlineService, ComplaintDeadlineScheduler
DTOs	`dto/`	RegisterRequest, LoginRequest, ResetPasswordRequest, ComplaintRequest, ComplaintResponse, StatusUpdateRequest, UserResponse, AdminDashboardResponse
Entities	`entities/`	User, Complaint, Category, EmailOtp
Repositories	`repository/`	UserRepository, ComplaintRepository, CategoryRepository, EmailOtpRepository
Enums	`Enums/`	Role (STUDENT, ADMIN), ComplaintStatus (5 states), ComplaintPriority (LOW, MEDIUM, HIGH)
Configuration	`config/`	SecurityConfig, JwtUtil, JwtAuthenticationFilter, AuthorizationUtil, GlobalExceptionHandler, WebConfig, SupabaseStorageProperties, DatabaseMigrationRunner

API Endpoints Specification

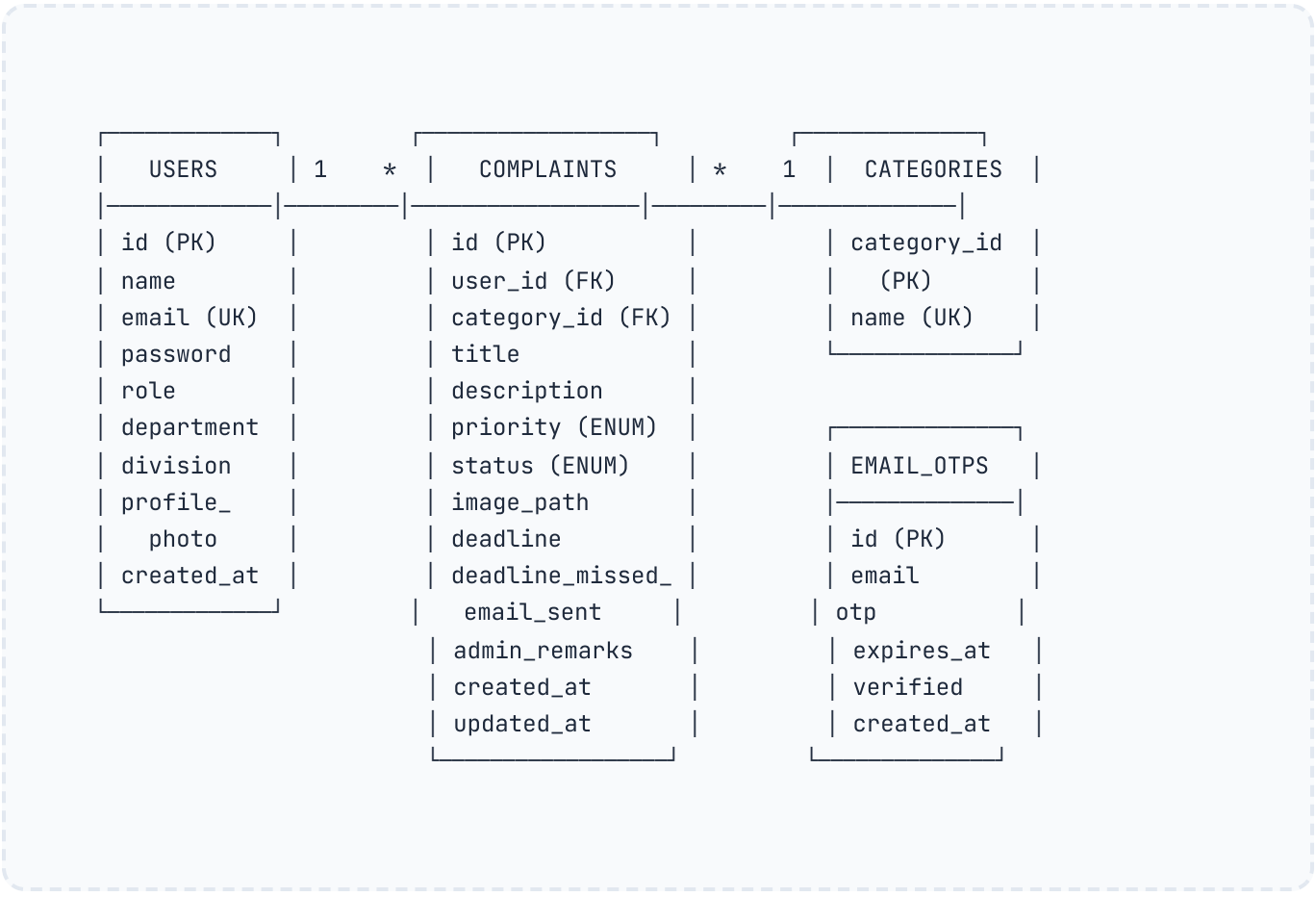
METHOD	ENDPOINT	PURPOSE	AUTH
GET	`/`	Health check	No
POST	`/api/auth/register`	Register new student with OTP + optional photo	No
POST	`/api/auth/send-otp`	Dispatch 6-digit registration OTP	No
POST	`/api/auth/login`	Authenticate and return JWT token	No
POST	`/api/auth/forgot-password/send-otp`	Dispatch password reset OTP	No
POST	`/api/auth/forgot-password/reset`	Verify OTP and reset password	No
GET	`/api/auth/test`	Connectivity test	No
POST	`/api/complaints/{userId}`	Submit complaint with photo	Student + ID
GET	`/api/complaints/getMyComplaint/{userId}`	Get student's complaints	Student + ID
GET	`/api/complaints/categories`	Get all categories	Yes
GET	`/api/users/getProfile/{userId}`	Get student profile	Yes + ID
POST	`/api/users/{userId}/photo`	Upload/update profile photo	Yes + ID
GET	`/api/admin/dashboard`	Admin dashboard statistics	Admin
GET	`/api/admin/complaints`	Paginated complaint list	Admin
GET	`/api/admin/Allcomplaints`	Complete complaint list	Admin
GET	`/api/admin/complaints/{id}`	Complaint detail	Admin
PUT	`/api/admin/complaints/{id}/status`	Update status with remarks	Admin

5 Database Analysis

Database Technology & Configuration

- **Engine**: MySQL 8.x managed instance on **Aiven Cloud**
- **Database Name**: `college_complaint_db` (env: `AIVEN_DB`)
- **Connection**: JDBC over SSL (`jdbc:mysql://...?sslmode=required`)
- **Schema Management**: Hibernate `ddl-auto: update` + custom `DatabaseMigrationRunner.java`

Entity Relationship Diagram



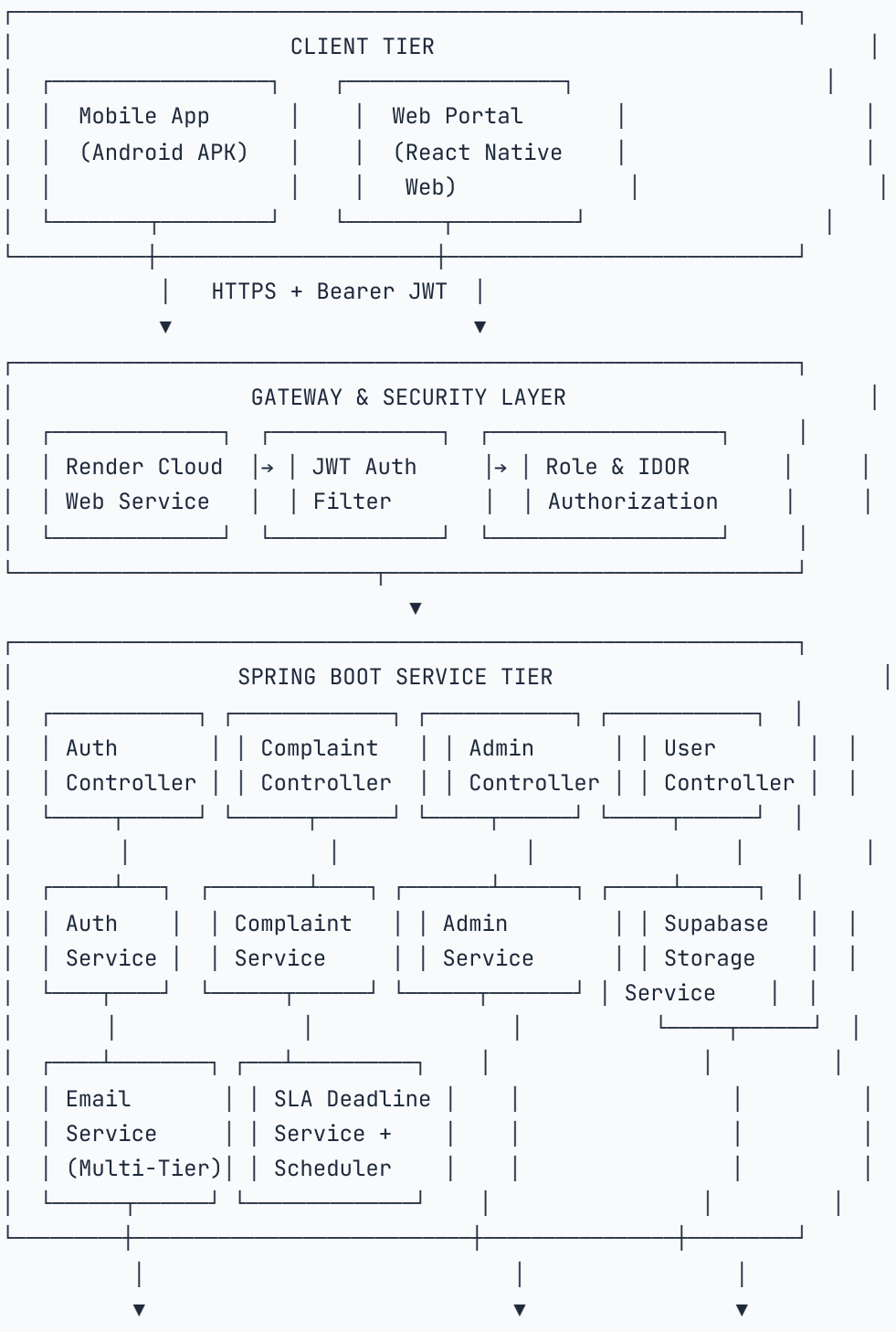
Database Structure Table

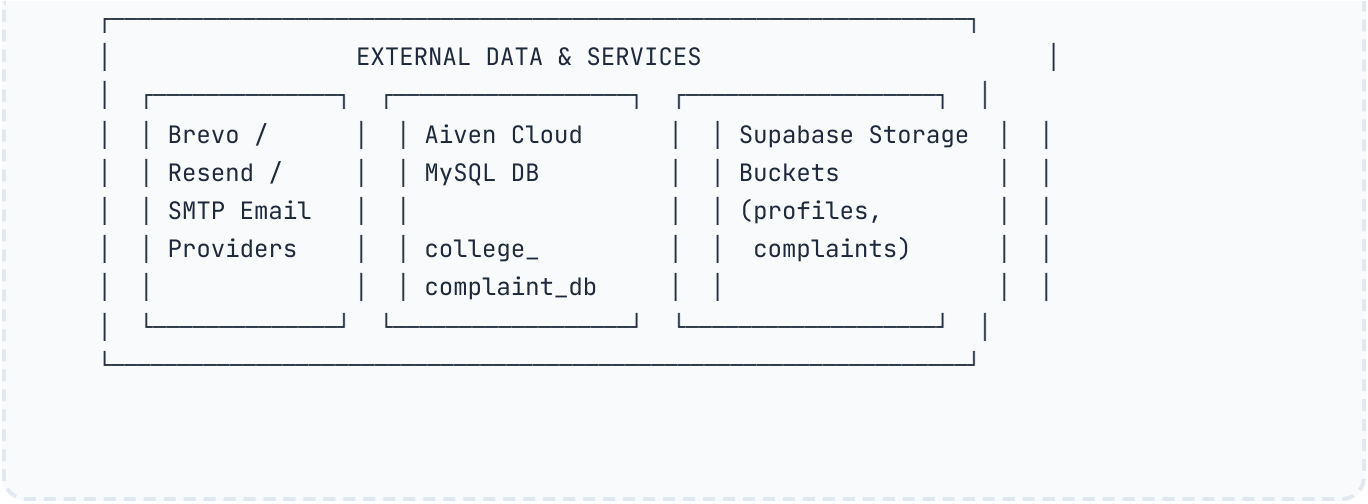
TABLE / ENTITY	IMPORTANT FIELDS	RELATIONSHIPS	PURPOSE
users `User.java`	`id` (PK, Auto-Inc), `name`, `email` (Unique), `password` (BCrypt),	One-to-Many → complaints	Stores credentials, institutional roles,

TABLE / ENTITY	IMPORTANT FIELDS	RELATIONSHIPS	PURPOSE
	`role` (STUDENT/ADMIN), `department`, `division`, `profile_photo` (Supabase URL), `created_at`		academic departments, and profile photos
categories `Category.java`	`category_id` (PK, Auto-Inc), `name` (Unique)	One-to-Many → complaints	Predefined complaint departments/domains
complaints `Complaint.java`	`id` (PK), `user_id` (FK → users), `category_id` (FK → categories), `title`, `description` (TEXT), `priority` (LOW/MEDIUM/HIGH), `status` (5 states), `image_path`, `deadline`, `deadline_missed_email_sent`, `admin_remarks`, `created_at`, `updated_at`	Many-to-One → users, categories	Core table tracking every student grievance, SLA deadline, and resolution
email_otps `EmailOtp.java`	`id` (PK), `email`, `otp` (6-digit), `expires_at` (5 min), `verified`, `created_at`	Standalone (transient)	Tracks OTP codes for registration and password resets

6 System Architecture & Workflow

System Architecture Diagram





Key User Workflows

Workflow 1: Student Registration with OTP Verification

1. **User action**: Student enters name, `@scet.ac.in` email, department, division, password, and taps "Send OTP".
2. **Frontend processing**: `register.tsx` validates the domain using regex and sends POST to `/api/auth/send-otp`.
3. **Backend validation**: `AuthService` verifies the email is not already registered, generates a secure 6-digit code.
4. **Database & external service**: Saves OTP record in `email_otps` table (5-min expiry). `EmailService` dispatches HTML email via Brevo/Resend.
5. **Response**: Returns HTTP 200 `{"message": "Verification code sent to …"}`.
6. **User completes**: Frontend reveals OTP input, student enters code and optional photo, then taps "Register". Backend verifies OTP, encodes password with BCrypt, saves user to `users` table, and uploads photo to Supabase.
7. **Final result**: Success alert displayed; student is routed to the login screen.

Workflow 2: Complaint Submission with Photo & SLA Calculation

1. **User action**: Student fills complaint form, selects category/priority, captures photo with camera, taps "Submit".
2. **Frontend processing**: `addComplaint.tsx` validates all fields, packages into `FormData`, attaches JWT.
3. **API request**: POST `/api/complaints/{userId}` with `multipart/form-data`.
4. **Backend processing**: `AuthorizationUtil` verifies userId matches token. `ComplaintService` loads user and category entities.
5. **SLA & Storage**: `WorkingHoursDeadlineService` computes deadline (e.g., 12 working hours for HIGH). `SupabaseStorageService` uploads photo to cloud.
6. **Database**: Complaint persisted in `complaints` table with status `ASSIGNED`, deadline, and image URL.
7. **Final result**: Success alert shown; student dashboard refreshes with new complaint card.

Workflow 3: Admin Review, Status Progression & Notification

1. **User action**: Admin opens complaint details, inspects photo evidence, selects next status (e.g., "RESOLVED"), enters remarks.
2. **Frontend processing**: `admin/complaintDetails.tsx` validates remarks (required for rejection), sends PUT request.
3. **API request**: PUT `/api/admin/complaints/{id}/status` with JSON payload.
4. **Backend validation**: `AdminService` validates transition against the state machine (e.g., IN_PROGRESS → RESOLVED is valid).
5. **Database & Email**: Updates `status`, `admin_remarks`, and `updated_at`. `EmailService` sends formatted HTML notification to the student.
6. **Response**: Returns updated complaint data.
7. **Final result**: Detail view reflects "RESOLVED" badge; student receives email notification.

7 Four-Member Team Division

> **Note on Authorship:** Git commit history shows commits under a single contributor. The following assignments represent a **proposed team division** of logical responsibilities. Individual ownership cannot be verified without separate task records.

Frontend Developer

Member 1 — UI & User Experience

Focus: Visual Design System, Theme Engine, Animated Splash Screen, Screen Layouts, Responsive Breakpoints, Navigation, Component Styling

Key Files:

- `frontend/constants/theme.ts` — Light & Dark design tokens, color palettes
- `frontend/constants/responsive.ts` — Mobile/Tablet/Desktop breakpoint hooks
- `frontend/context/ThemeContext.tsx` — Global theme state & persistence provider
- `frontend/app/index.tsx` — SVG animated splash screen (6-sec circular progress)
- `frontend/app/_layout.tsx` — App root layout, ThemeProvider, SafeAreaProvider
- `frontend/components/ThemeToggle.tsx` — Dark/Light toggle button component
- `frontend/components/themed-text.tsx`, `themed-view.tsx` — Themed reusable containers

Responsibilities: Establish and maintain UI design tokens. Implement SVG animations and staged typography. Design responsive card layouts, status badges, and navigation headers. Implement dark/light theme switching with persistence.

Frontend Developer

Member 2 — API Integration & Client-Side Logic

Focus: Form Handling, Client Validations, Camera/Photo Integration, API Interceptor, JWT Storage, Session Management, Admin Search/Filter Logic, Error Handling

Key Files:

- `frontend/constants/api.ts` — `apiFetch` wrapper, API_BASE_URL resolution, token injection, session expiry
- `frontend/utils/authStorage.ts` — Platform-aware JWT storage (SecureStore / localStorage / AsyncStorage)
- `frontend/app/login.tsx` — Login form, credential validation, role-based routing
- `frontend/app/register.tsx` — OTP flow, countdown timer, multipart registration
- `frontend/app/forgotPassword.tsx` — Password reset OTP flow
- `frontend/app/addComplaint.tsx` — Complaint form, multipart image upload, camera integration
- `frontend/app/myComplaints.tsx` — Complaint list data fetching & display
- `frontend/app/profile.tsx` — Profile fetch & photo upload logic
- `frontend/app/admin/complaints.tsx` — Client-side search, filter, & pagination
- `frontend/app/admin/complaintDetails.tsx` — Status update PUT request logic

Responsibilities: Build form state logic, OTP countdown timers, and field validations. Implement platform-aware token storage. Build the `apiFetch` interceptor. Handle multipart form packaging for image uploads across web and mobile.

Backend Developer

Member 3 — APIs & Business Logic

Focus: REST Controller Endpoints, Spring Security Filter Chain, JWT Validation, Role-Based Access, IDOR Guards, State Machine Rules, Working-Hours SLA Engine, Scheduled Daemon

Key Files:

- `AuthController.java` — Registration, login, OTP, and password reset endpoints
- `ComplaintController.java` — Student complaint submission and retrieval endpoints
- `AdminController.java` — Admin dashboard, listing, and status update endpoints
- `UserContoller.java` — User profile and photo upload endpoints
- `SecurityConfig.java` — SecurityFilterChain, CORS, URL-based permissions
- `JwtUtil.java` — HMAC-SHA256 JWT token generator and claim extractor
- `JwtAuthenticationFilter.java` — OncePerRequestFilter for Bearer token validation
- `AuthorizationUtil.java` — Horizontal access (IDOR) security verification
- `GlobalExceptionHandler.java` — Centralized REST exception handler
- `AdminService.java` — Status transition validation, rejection rules, email triggers
- `AuthService.java` — Registration logic, OTP verification, login, password reset
- `WorkingHoursDeadlineService.java` — SLA calculation algorithm (9 AM–5 PM, Mon–Fri)
- `ComplaintDeadlineScheduler.java` — Background 60-second daemon for SLA monitoring

Responsibilities: Design and implement all REST controller endpoints. Configure stateless Spring Security with JWT and role authorization. Develop the working-hours SLA calculation algorithm. Enforce complaint status lifecycle transitions. Build the scheduled deadline monitoring daemon.

Backend Developer

Member 4 — Database & Cloud Integrations

Focus: Database Entities, Relationships, JPA Repositories, JPQL Queries, Database Migration, Supabase Cloud Storage, Multi-Provider Email, Environment Configuration, Docker

Key Files:

- `User.java` — JPA entity for users table
- `Complaint.java` — JPA entity for complaints table with lifecycle hooks
- `Category.java` — JPA entity for categories table
- `EmailOtp.java` — JPA entity for email_otps table
- `ComplaintRepository.java` — Custom JPQL queries for sorting and overdue detection
- `UserRepository.java`, `CategoryRepository.java`, `EmailOtpRepository.java`
- `DatabaseMigrationRunner.java` — Startup schema verification (ALTER TABLE)
- `SupabaseStorageService.java` — Cloud asset upload, deletion via REST API
- `SupabaseStorageProperties.java` — Typed config properties for Supabase
- `EmailService.java` — Multi-tier email dispatcher (Brevo → Resend → SMTP)
- `ComplaintService.java` — Business logic for complaint creation & retrieval
- `application.yml` — Spring Boot central configuration
- `DockerFile` — Multi-stage build for deployment
- `.env` — Environment variable definitions

Responsibilities: Design JPA entities, constraints, primary/foreign keys. Write optimized JPQL queries. Build Supabase Storage integration. Construct multi-provider email dispatcher. Maintain database migrations and containerization configurations.

Team Contribution Summary

MEMBER	ROLE	MODULES	KEY RESPONSIBILITIES
Member 1	Frontend: UI/UX	Theme System, Splash Screen, Responsive Layouts, Navigation, Component Styling	Design tokens, animations, responsive UI, dark/light mode
Member 2	Frontend: Logic	API Interceptor, Auth Storage, Forms, Validations, Camera, Search/Filter	API integration, JWT management, form logic, multipart uploads
Member 3	Backend: APIs	Controllers, Security, JWT, Authorization, SLA Engine,	REST endpoints, security configuration, business rules, state

MEMBER	ROLE	MODULES	KEY RESPONSIBILITIES
		Scheduler	machine
Member 4	Backend: DB & Integration	Entities, Repositories, Supabase Storage, Email Service, Docker	Database design, cloud storage, email integration, deployment

8 Project Folder Structure

```
e:/StudentComplaint/
├── backend/                          # Spring Boot 4.1.0 Java 21 Application
│   ├── .env                          # Environment variable definitions
│   ├── DockerFile                    # Multi-stage Docker build
│   ├── pom.xml                       # Maven dependencies & plugins
│   ├── src/main/
│   │   ├── java/com/project/ComplaintApp/
│   │   │   ├── ComplaintAppApplication.java  # Entry point, .env loader
│   │   │   ├── config/               # Security, JWT, storage, exceptions
│   │   │   │   ├── AuthorizationUtil.java
│   │   │   │   ├── DatabaseMigrationRunner.java
│   │   │   │   ├── GlobalExceptionHandler.java
│   │   │   │   ├── JwtAuthenticationFilter.java
│   │   │   │   ├── JwtUtil.java
│   │   │   │   ├── SecurityConfig.java
│   │   │   │   ├── SupabaseStorageProperties.java
│   │   │   │   └── WebConfig.java
│   │   │   ├── controller/           # REST API Controllers
│   │   │   │   ├── AdminController.java
│   │   │   │   ├── AuthController.java
│   │   │   │   ├── ComplaintController.java
│   │   │   │   ├── controller.java
│   │   │   │   └── UserContoller.java
│   │   │   ├── dto/                  # Data Transfer Objects
│   │   │   ├── entities/             # JPA Entities (User, Complaint, ...)
│   │   │   ├── Enums/                # Role, ComplaintStatus, Priority
│   │   │   ├── repository/           # Spring Data JPA Repositories
│   │   │   └── services/             # Business Logic Services
│   │   │       ├── AdminService.java
│   │   │       ├── AuthService.java
│   │   │       ├── ComplaintDeadlineScheduler.java
│   │   │       ├── ComplaintService.java
│   │   │       ├── EmailService.java
│   │   │       ├── SupabaseStorageService.java
│   │   │       └── WorkingHoursDeadlineService.java
│   │   └── resources/
│   │       └── application.yml       # Spring configuration
│   └── uploads/                      # Local upload fallback
│
├── frontend/                         # Expo SDK 57 React Native App
│   ├── .env                          # Client environment config
│   ├── app/                          # File-based navigation screens
│   │   ├── _layout.tsx               # Root layout (ThemeProvider, Stack)
```

```
│   │   ├── index.tsx                  # Animated splash screen
│   │   ├── login.tsx                  # Authentication screen
│   │   ├── register.tsx               # Student registration
│   │   ├── forgotPassword.tsx         # Password reset
│   │   ├── dashboard.tsx              # Student dashboard
│   │   ├── addComplaint.tsx           # Complaint submission form
│   │   ├── myComplaints.tsx           # Student complaint history
│   │   ├── profile.tsx                # Profile & avatar management
│   │   └── admin/
│   │       ├── dashboard.tsx          # Admin metrics dashboard
│   │       ├── complaints.tsx         # Admin complaint list
│   │       └── complaintDetails.tsx   # Admin complaint detail & update
│   ├── components/                    # Reusable UI widgets
│   ├── constants/                     # api.ts, responsive.ts, theme.ts
│   ├── context/                       # ThemeContext.tsx
│   ├── hooks/                         # Custom React hooks
│   ├── utils/                         # authStorage.ts
│   ├── eas.json                       # EAS Build config
│   └── package.json                   # Dependencies & scripts
│
└── uploads/                           # Root upload directory fallback
```

9 Security and Configuration

Authentication & Authorization Mechanisms

- **Stateless Bearer JWT**: Authenticated endpoints validate signed HMAC-SHA256 tokens via `JwtAuthenticationFilter.java`.
- **Role-Based Guards**: `/api/admin/**` requires `ROLE_ADMIN`; `/api/complaints/**` and `/api/users/**` require authentication.
- **IDOR Protection**: `AuthorizationUtil.java` compares the `userId` in route path variables with the user ID embedded in the JWT claims.
- **Client Session Isolation**: Uses `expo-secure-store` (hardware-backed encryption) on native mobile devices.

Input Validation & Protection

- **Domain Verification**: `AuthService.java` validates email structure and enforces allowed domains (`scet.ac.in`).
- **SQL Injection Prevention**: Spring Data JPA uses parameterized JPQL queries.
- **Password Security**: BCrypt with automatic salt generation. Passwords excluded from JSON via `@JsonIgnore`.

Environment Variables

CATEGORY	VARIABLE NAMES (VALUES NOT SHOWN)
Database	`AIVEN_HOST`, `AIVEN_PORT`, `AIVEN_DB`, `AIVEN_USERNAME`, `AIVEN_PASSWORD`
Object Storage	`SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`
JWT	`JWT_SECRET_KEY`
Email Services	`BREVO_API_KEY`, `BREVO_FROM_EMAIL`, `RESEND_API_KEY`, `RESEND_FROM_EMAIL`, `EMAIL_USERNAME`, `EMAIL_PASSWORD`, `MAIL_HOST`, `MAIL_PORT`
Server	`SERVER_PORT`
Frontend	`EXPO_PUBLIC_API_BASE`

Identified Security Issues

⚠️ **Permissive CORS (Confirmed Risk)**: `SecurityConfig.java` allows all origins (`*`). In production, this should be restricted to the specific frontend domain.

⚠️ **Client-Side API Key Exposure (Confirmed Risk)**: `frontend/.env` contains a `RESEND_API_KEY`. Client-side builds can expose bundled environment variables; email service keys must reside solely on the backend.

⚡ **Hardcoded Fallback IP (Medium Risk)**: `api.ts` falls back to `http://10.118.31.1:10000` over plain HTTP if `EXPO_PUBLIC_API_BASE` is unset.

⚡ **Verbose Server-Side Logging (Low Risk)**: Full JWT claims and user emails are printed to stdout in controllers and filters, which may appear in production application logs.

10 Missing Features & Improvements

Incomplete or Partially Implemented Features

- **Unused Starter Expo Tabs**: `app/(tabs)/explore.tsx` and `(tabs)/index.tsx` are boilerplate artifacts from `create-expo-app` and are not linked in the main workflows.
- **Local vs Cloud Storage Inconsistency**: `WebConfig.java` configures `/uploads/**` static handlers on the local filesystem, but services upload exclusively to Supabase Storage. The local handler is essentially legacy code.
- **Complaint Re-opening / Student Feedback**: Once an administrator marks a complaint as RESOLVED or REJECTED, the student cannot dispute or provide a satisfaction rating.

Recommended Improvements

PRIORITY	AREA	RECOMMENDATION
HIGH	Security	Remove `RESEND_API_KEY` from `frontend/.env`. Restrict CORS in `SecurityConfig.java` from `*` to the deployed frontend origin. Rotate all exposed credentials.
HIGH	Security	Add `.env` to `.gitignore` across both frontend and backend to prevent committing credentials.
MEDIUM	Cleanup	Remove unused starter routes in `frontend/app/(tabs)` to reduce confusion and bundle size.
MEDIUM	Features	Implement a student feedback mechanism (1–5 star ratings) or complaint reopen workflow within 48 hours of resolution.
MEDIUM	Admin	Add batch status updates and CSV/PDF export capabilities for institutional compliance audits.
LOW	Testing	Write automated unit and integration tests for Spring Boot controllers (`@WebMvcTest`) and React Native screens using Jest.
LOW	Observability	Replace `System.out.println` statements with structured logging via SLF4J (`@Slf4j`).

11 Final Project Summary

Project Description (for College Presentation)

> *"The College Complaint Management System is a full-stack mobile and web platform that digitizes and automates student grievance redressal at Sarvajanik College of Engineering & Technology. It replaces manual complaint handling with institutional email authentication, photographic evidence uploads via Supabase cloud storage, automated working-hours SLA deadline tracking, multi-tier email updates, and an administrative oversight dashboard — providing complete transparency from submission to resolution."*

Major Implemented Features

1. Institutional email domain gating (`@scet.ac.in`) with 6-digit OTP verification
2. Role-based authentication using HMAC-SHA256 JWT tokens (24-hour expiry)
3. Complaint lodgment with category classification, priority levels (LOW/MEDIUM/HIGH), and camera/gallery attachments
4. Working-hours SLA calculation engine (12h / 24h / 48h) accounting for evenings and weekends
5. Automated background daemon (60-second interval) for SLA breach detection and email alerts
6. Multi-provider transactional email system (Brevo → Resend → SMTP failover)
7. Administrative triage dashboard with search, filtering, pagination, and status transition workflows
8. User profile management with real-time avatar customization via cloud storage
9. Light and dark theme engine with persistent state across web and mobile platforms
10. Status lifecycle state machine with mandatory admin remarks on rejection

Complete Technology Stack

LAYER	TECHNOLOGIES
Frontend	React 19, React Native 0.86, Expo SDK 57, Expo Router, TypeScript 6.0, Reanimated, Expo SecureStore, Expo Image
Backend	Java 21, Spring Boot 4.1.0 (WebMVC, Data JPA, Security, Mail, Validation), JJWT 0.12.5, Lombok
Datastore	MySQL 8.x (Aiven Cloud), Supabase Storage REST API

LAYER	TECHNOLOGIES
Email	Brevo REST API, Resend REST API, Jakarta Mail / SMTP
Infrastructure	Docker multi-stage build, Render Cloud Web Service, EAS Build (Android APK)

System Integration Summary

The Expo React Native frontend connects to the Spring Boot REST API through an `apiFetch` client wrapper that automatically manages Bearer tokens and handles session expiry. The backend parses incoming requests, authenticates them against a stateless JWT security filter, verifies horizontal authorization to prevent IDOR vulnerabilities, and routes requests to appropriate controllers. Business logic calculates deadlines using working-hour rules and offloads images to Supabase Storage. Entity state is persisted in an Aiven MySQL database, and background schedulers trigger transactional emails through Brevo or Resend to keep students informed of progress.

Four-Member Team Responsibilities Summary

MEMBER	ROLE	CORE RESPONSIBILITY
Member 1	Frontend: UI/UX	Design tokens, themes, splash animation, responsive layouts, navigation
Member 2	Frontend: Logic	API integration, JWT storage, forms, validations, camera, search/filter
Member 3	Backend: APIs	REST endpoints, security, authorization, SLA engine, scheduler daemon
Member 4	Backend: DB	Entity design, repositories, Supabase storage, email service, Docker

Top Improvements Needed

1. **Security Hardening**: Remove client-exposed API keys, restrict CORS origins, add `.env` to `.gitignore`.
2. **Template Cleanup**: Remove unused boilerplate routes in `app/(tabs)`.
3. **Student Feedback Loop**: Implement resolution feedback ratings and complaint reopen workflow.
